Midterm Project: Machine Learning Pipeline, Hyperparameter Tuning, and Ensemble Learning

Charles Omaoeng, Joel Villarreal

March 27th, 2026

1 Introduction

Our implementation is organized around a `Pipeline` class that manages dataset selection, preprocessing, and classifier training/testing. The pipeline supports both the `MNIST` and `FashionMNIST` datasets, and allows preprocessing with either PCA or LDA before classification with a `scikit-learn` SVC.

Main Pipeline:

Step 1: Load MNIST and FashionMNIST datasets (both training/testing).

Step 2: Flatten and standardize both datasets.

Step 3: Reduce the dimensionality of both datasets using PCA and LDA.

Step 4: Save reduced datasets for later usage.

Step 5: Create `scikit-learn` SVC with a chosen kernel and hyperparameter settings.

Step 6: Train the SVC on a chosen reduced training dataset.

Step 7: Evaluate the trained SVC using the corresponding testing dataset.

This design allows the same pipeline structure to be reused across different datasets, dimensionality reduction methods, and kernel choices. This makes testing different hyperparameter and dimensionality reduction combinations more organized.

2 Best Hyper Parameter Settings

Present the best hyperparameter settings you found via tuning and explain how are they obtained.

	Linear (MNIST)	RBF (MNIST)	Linear (Fashion)	RBF (Fashion)
C	0.01	10	0.01	1
gamma	N/A	0.001	N/A	0.01

Table 1: Best Hyperparameter Settings for Linear and RBF Kernels For Both the MNIST & Fashion MNIST Datasets, For all PCA Reductions

The best hyperparameter settings found via tuning are shown in Table 1, Table 2, and Table 3. These values were obtained by running each dataset, with each Support Vector Classifier (SVC) kernel, for variable C, gamma, and degree values where appropriate, and with a maximum of 512 iterations. This was a slow process, which involved increasing the max number of iterations to train a model, and decreasing the range of viable C, gamma, and degree values, until a desirable number of iterations was met, from which we determined the best hyperparameter setting. The "best" behavior was based on accuracy of the generated

PCA reduction	C	gamma	degree
50	0.01	0.1	2
100	1	0.01	2
200	1	0.01	2

Table 2: Best Hyper Parameter Settings for Poly Kernel for MNIST Dataset

PCA reduction	C	gamma	degree
50	1	0.01	2
100	1	0.1	3
200	1	1	3

Table 3: Best Hyper Parameter Settings for Fashion MNIST Dataset

model at labeling the test data, for more information on this see Section 3.2. The algorithm outlined in Algorithm 1 demonstrates how the best hyper-parameters can be found for the polynomial case, but the Linear and RBF can be similarly stated, with less nested looping due to having less hyper-parameters. Notice that Algorithm 1 would need to be run four times for the various reductions, and two times per reduction for the two datasets, a total of eight times to find the best hyperparameter settings for all potential initial datasets.

Notice that for a linear SVC kernel, the best performing hyper parameter settings for the MNIST data set and Fashion data set were found to be equivalent, as shown in Table 1. Additionally, for the Linear and Radial Bias Function (RBF) kernels, the best hyperparameter values were independent of the number of components left after a Principal Component Analysis (PCA) transformation was performed, which is why Table 1 does not have distinct parameter values for each PCA reduction. Contrastingly, an SVC with a polynomial kernel trained for 512 iterations did have unique best-performing hyperparameter settings for the various PCA reductions, as shown in Table 3 and Table 2.

Something else of note about our polynomial findings for the fashion MNIST data set was that the C-value of the polynomial kernel was relatively insignificant compared to the other two hyper parameter settings. More specifically, for a PCA reduction of 100, a gamma of 0.1, and degree of 0.3 performed best, but the C values in the set $\{0.01, 0.1, 1, 10\}$ all performed equally well; and with a PCA reduction of 200, the C values in the set $\{0.001, 0.01, 0.1, 1, 10\}$ all performed equally well. Note that this was not the case with a PCA reduction of 50, nor was it the case for the MNIST dataset.

Algorithm 1: PolyLoopX, y, X_t, y_t, C, G, D

Input:
C - C values
G - gamma values
D - degree values
X - train data (preprocessed)
y - train labels
X_t - test data (preprocessed)
y_t - test labels
Output: best performing C, gamma, degree combination

```
best_acc ← 0
best ← (0, 0, 0)
for c in C do
    for gamma in G do
        for degree in D do
            S ← an SVC with a polynomial kernel with the hyper-parameters c, gamma, and degree,
              with a maximum number of iterations of 512
            S.fit(X, y)
            acc ← accuracy of predicting the labels of the test data X_t
            if acc > best_acc then
                best ← (c, gamma, degree)
return best
```

3 Machine Learning Pipeline (Task 3)

3.1 PCA vs. LDA

PCA vs. LDA. The comparison should be based on the time cost of training, including the dimensionality reduction, and the test error. You may present a table (for each dataset) which includes all three reduced scales of PCA and LDA.

The performance of the three SVC kernels (based on training times and prediction errors) generated for both datasets using PCA and LDA are listed in Table 4 and Table 5. The information below was the output of testing using the hyper parameters specified in the previous section.

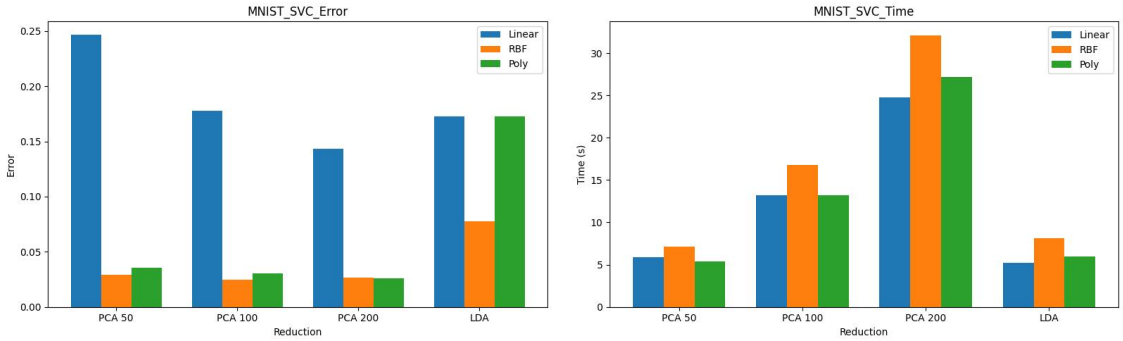
On MNIST, the training time of SVC increases as the number of PCA components increase from 50 to 100, then to 200 for all three kernels. Despite that, increasing PCA dimensions generally reduces testing error. This is because higher dimensionality preserves more of the original structure and variance of the original dataset at the cost of increasing training time (due to having more data to consider), and the results from the classifier displays this trend. LDA exhibits much shorter times compared to the three reduced dimensionalities. This is most likely due to the fact that LDA produces a lower dimensional representation than the larger number of components with PCA 100 and 200. A drawback though is that it lacks in accuracy when compared to PCA. This is especially evident when looking at the observed LDA errors and comparing them to PCA 100 and PCA 200, where PCA achieves noticeably lower test error at the cost of higher computational time.

On Fashion-MNIST, a similar increase in training time is observed as the number of PCA components increase. More components generally improve the test performance, but it greatly increases the total computational cost. Compared to PCA, LDA once again produces much shorter training times. Unlike with MNIST though, LDA manages to achieve a much lower test error than the corresponding PCA settings for the linear and polynomial kernels while also requiring less time. For the RBF kernel, PCA remains superior in that regard. LDA might perform better because of the label information for the much more difficult Fashion-MNIST dataset.

Across both datasets, a clear tradeoff between computation time and predictive performance can be observed. LDA is consistently faster across both datasets, but it struggles in test accuracy for certain kernels, and performance mainly depends on the dataset. In the case of MNIST, LDA is worse than PCA whereas on Fashion-MNIST, LDA is shown to be similar or even better in performance than PCA.

	PCA 50			PCA 100			PCA 200			LDA		
	Linear	RBF	Poly	Linear	RBF	Poly	Linear	RBF	Poly	Linear	RBF	Poly
time (s)	5.85	7.16	5.37	13.20	16.81	13.22	24.73	32.11	27.15	5.23	8.12	5.94
error	0.2466	0.0289	0.0356	0.1780	0.0248	0.0302	0.1433	0.0265	0.0259	0.1726	0.0778	0.1725

Table 4: MNIST Dataset Kernel Performance

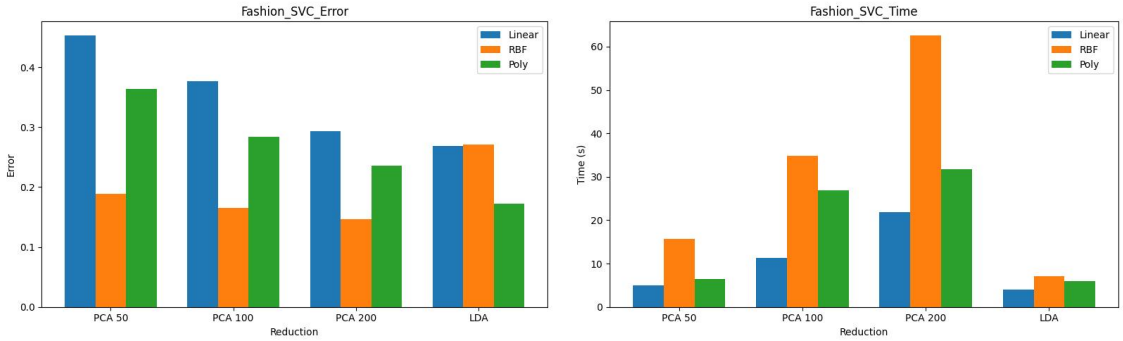


(a) Error (b) Times

Figure 1: MNIST Comparison

	PCA 50			PCA 100			PCA 200			LDA		
	Linear	RBF	Poly	Linear	RBF	Poly	Linear	RBF	Poly	Linear	RBF	Poly
time (s)	5.05	15.77	6.52	11.36	34.89	26.89	21.91	62.65	31.69	4.03	7.16	5.94
error	0.4534	0.1889	0.3633	0.3768	0.1649	0.2838	0.2929	0.1459	0.236	0.2688	0.2704	0.1725

Table 5: Fashion MNIST Dataset Kernel Performance



(a) Error (b) Times

Figure 2: Fashion MNIST Comparison

3.2 Three kernels in SVC

Three kernels in SVC. You may only consider the 3 PCA case and compare the training time (only the time of training the SVC) and the prediction error on the test dataset. Please use a table (for each dataset) with explanations.

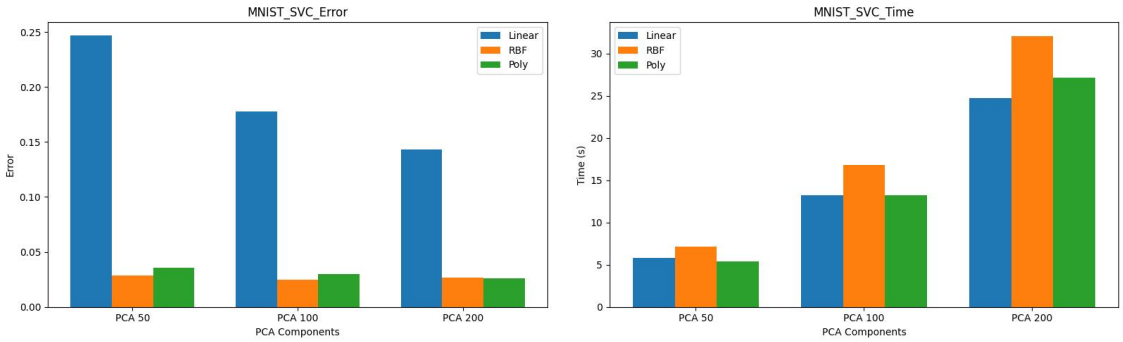
The performance of the three SVC kernels generated from the three PCA reductions for both datasets have training times and prediction errors which are listed in Table 6 and Table 7. For an easier comparison, please reference Figure 3 and Figure 4 respectively. A note on the error and timings: they were generated from only those models with hyper parameters listed in Section 2, for 512 iterations.

Findings: There is a clear tradeoff in training SVC models, where the best performing models, take the longest to train. Below is a brief synopsis, followed by a more detailed analysis.

- The linear kernel type always had the highest number of misclassifications, with RBF having the least in the majority of the cases. However, the RBF kernel took longest to train, while the linear was fastest.
- Increasing the number of components in a dataset reduction corresponded to a reduction in the number of misclassifications. However, increasing the number of components in a dataset reduction also corresponded to an increase in model training time.

	PCA 50			PCA 100			PCA 200		
	Linear	RBF	Poly	Linear	RBF	Poly	Linear	RBF	Poly
time (s)	5.85	7.16	5.37	13.20	16.81	13.22	24.73	32.11	27.15
error	0.2466	0.0289	0.0356	0.1780	0.0248	0.0302	0.1433	0.0265	0.0259

Table 6: MNIST Dataset Kernel Performance



(a) Error (b) Times

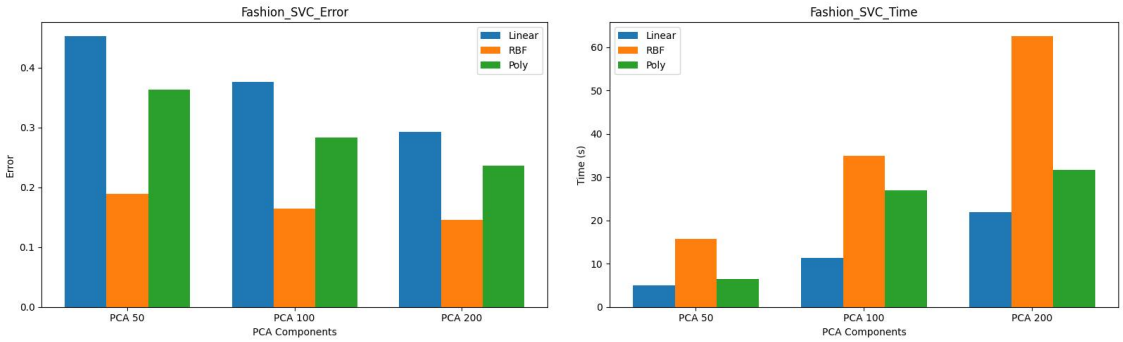
Figure 3: MNIST Comparison

MNIST Error Analysis: Notice that for the MNIST dataset, the kernel type played the most significant role in reducing error. Where the relative error for the RBF kernel trained on a PCA 100 and PCA 200 dataset, as well as the relative error of the polynomial kernel trained on PCA reductions of 100 and 200, are all low compared to the error which occurs when choosing a linear kernel (See Figure 3). When comparing those best performing kernels alone (i.e. ignoring the linear case), the model trained with an RBF kernel performed best (in terms of error) for both the PCA 50 and 100 case, while the polynomial kernel performed best for the PCA 200 case.

MNIST Time Analysis: Reduction type played the most significant role in reducing time costs. By referencing Figure 3, it is clear that relative time for training models based on the various kernels is not drastically different, compared to the speed up which occurs when decreasing the number of components after a PCA reduction. Additionally, when comparing time costs of training models with the various kernels,

	PCA 50			PCA 100			PCA 200		
	Linear	RBF	Poly	Linear	RBF	Poly	Linear	RBF	Poly
time (s)	5.05	15.77	6.52	11.36	34.89	26.89	21.91	62.65	31.69
error	0.4534	0.1889	0.3633	0.3768	0.1649	0.2838	0.2929	0.1459	0.236

Table 7: Fashion MNIST Dataset Kernel Performance



(a) Error

(b) Times

Figure 4: Fashion MNIST Comparison

the model trained with an RBF kernel was always the slowest, while the cost of training a linear model was always fastest.

Fashion MNIST Error Analysis: For the Fashion MNIST dataset, the strongest determinant of error was kernel type. By referencing Figure 4, it is clear that the relative error of variable reduction sizes is not as significant as the variance in error which occurs when switching the model's kernel type. More succinctly, the RBF model had significant performance increase compared to poly and linear, with linear always performing worst. Additionally, it is still worth mentioning that as the PCA component amount increased, the error decreased, an intuitive result.

Fashion MNIST Time Analysis For the Fashion MNIST dataset, both reduction type and kernel type play a a significant role in training time. Referencing Figure 4, notice that the RBF kernel took significantly longer to train than the poly and linear kernels, with linear always completing fastest. Additionally, notice that this difference in timing is only worsened as the PCA component count increases. A simple example of this is that a model with a linear kernel trained with a dataset reduced to 50 components took 5 seconds to train, while a model with an RBF kernel trained with a dataset reduced to 200 components took over a minute.

4 SVC vs. Bootstrap Aggregating

For Task 4, please give a comparison between a single SVC (result of Task 3) and the result of bootstrap aggregating for each kernel. The comparisons should be based on training time and test error.

The performance of the three SVC kernels previously mentioned in section 3, compared to ensemble training via Bagging for both datasets have training times and prediction errors which are listed in Table 8 and Table 9. For an easier comparison, please reference Figures 5, 6, 7, and 8. A note on the error and timings: they were generated from only those models with hyper parameters listed in Section 2, for 512 iterations.

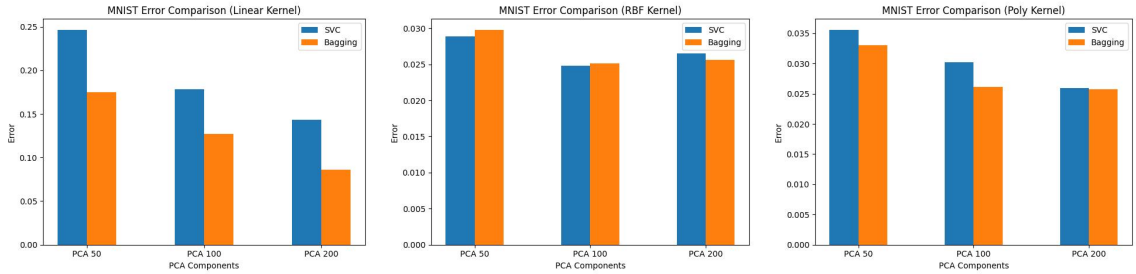
Findings: There is a clear tradeoff in training SVC models, where the best performing models, take the longest to train. Below is a brief synopsis, followed by a more detailed analysis.

- Models trained with bagging, for the vast majority of cases, outperformed their plain SVC counterparts. However, models with bagging took significantly longer to train.

- Models which had bagging and linear kernels saw the largest reduction in misclassifications. However, the linear kernel models had the most amount of error to begin with.
- Models which had bagging and RBF kernels saw the smallest reduction in misclassifications. However, the RBF kernel models had the least amount of error to begin with.

		PCA 50			PCA 100			PCA 200		
		Linear	RBF	Poly	Linear	RBF	Poly	Linear	RBF	Poly
SVC	time (s)	5.85	7.16	5.37	13.20	16.81	13.22	24.73	32.11	27.15
	error	0.2466	0.0289	0.0356	0.1780	0.0248	0.0302	0.1433	0.0265	0.0259
BAG	time (s)	57.71	65.71	46.47	115.08	138.47	106.37	188.11	232.19	190.31
	error	0.1748	0.0298	0.0330	0.1272	0.0251	0.0261	0.0861	0.0256	0.0257

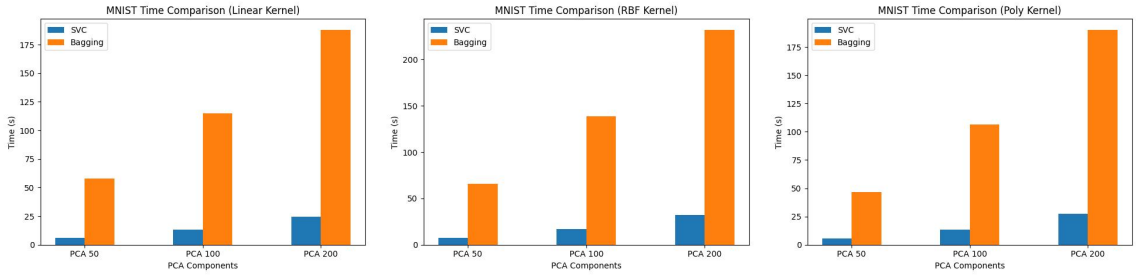
Table 8: SVC vs. BAGGING on MNIST Dataset



(a) Linear (b) RBF (c) Poly

Figure 5: MNIST Error Comparison

MNIST Error Analysis: By referencing Figure 5, it is clear that the models with bootstrap aggregating had lower prediction error than the regular SVC models, but there are some caveats. For one, notice that the RBF kernel specifically, did not see significant change in the number of misclassifications, and in the PCA 50 case, the Bagging model actually performed worse. However, with the linear classification, which performed the worst overall (see Section 3.2), the bagging had the most significant increase in performance. Moreover, notice that introducing bagging did not change the fact that linear kernel models behaved worst, and rbf kernel models behaved best.



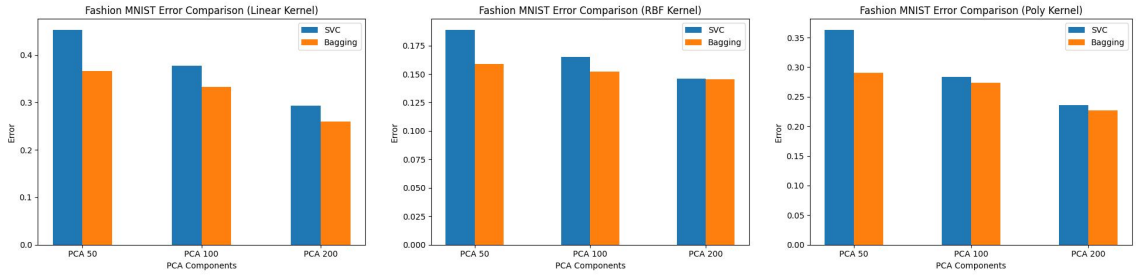
(a) Linear (b) RBF (c) Poly

Figure 6: MNIST Time Comparison

Fashion MNIST Error Analysis: By referencing Figure 7, it is clear that the models with bootstrap aggregating had lower prediction error than the regular SVC models in all cases. This decrease in prediction error is most significant for the PCA 50 case, which notably performed worst in general (see Section 3.2). With the improvement obtained via bagging becoming less substantial as the number of components in the

		PCA 50			PCA 100			PCA 200		
		Linear	RBF	Poly	Linear	RBF	Poly	Linear	RBF	Poly
SVC	time (s)	5.05	15.77	6.52	11.36	34.89	26.89	21.91	62.65	31.69
	error	0.4534	0.1889	0.3633	0.3768	0.1649	0.2838	0.2929	0.1459	0.2360
BAG	time (s)	43.22	124.78	43.34	88.14	274.14	93.35	169.56	488.13	191.27
	error	0.3659	0.1589	0.2906	0.3331	0.1521	0.2736	0.2598	0.1456	0.2274

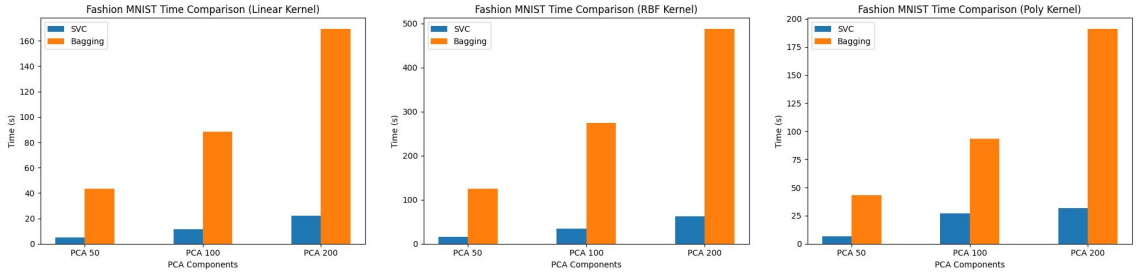
Table 9: SVC vs. BAGGING on Fashion MNIST Dataset



(a) Linear (b) RBF (c) Poly

Figure 7: Fashion MNIST Error Comparison

PCA reduction increases. Additionally, notice that the best performing model, one with an RBF kernel trained on a data set reduced to 200 dimensions, had the least amount of increase in performance due to bagging (though there is still an increase in performance). Moreover, notice that introducing bagging did not change the fact that linear kernel models behaved worst, and rbf kernel models behaved best.



(a) Linear (b) RBF (c) Poly

Figure 8: Fashion MNIST Time Comparison

Time Analysis: By referencing Figure 6 and Figure 8, it is clear that the models with bootstrap aggregating had significantly longer training times. This is true for all data set reduction types, for all kernel types. But it is worth noting, that this increase in model fitting time resulting from bagging is compounded as the number of PCA components increases, which is the expected behavior.